When and Why Does Personalized PageRank Fail on Code Graphs?
Analyzing Seed Dispersion, Graph Resolution Artifacts, and Decoupled Specificity Damping

Sultan Haikal
yvliet@users.noreply.github.com

October 2026

Abstract

Local developer agents must retrieve relevant code context within tight prompt budgets ($K \leq 4{,}096$ tokens). While Personalized PageRank (PPR) over code dependency graphs is commonly deployed to discover unmentioned symbols, its empirical efficacy is widely debated. Through a systematic 2×2 factorial evaluation on SWE-bench Lite ($N = 237$), I demonstrate that unconstrained PPR failure is driven primarily by **teleport prior dispersion** rather than scale-free hub absorption alone. Under diffuse linear priors, PPR dissipates probability mass across peripheral test helpers (32.5% test contamination), collapsing top-10 function accuracy to 10.1%. Concentrating query priors via top-25 softmax ($\tau = 0.15$) rescues standard PPR to 48.5%, while test-node graph sanitization adds +3.8 percentage points. **Perron** introduces post-walk Decoupled Specificity ($\boldsymbol{\pi}_q / (\boldsymbol{\pi}_g + \epsilon)^\gamma$), providing a closed-form topological stabilizer that rescues diffuse priors by +24.1 pp (34.2% vs. 10.1%) while restoring hub suppression to 99.5% (reducing hub contamination from 9.2% in Standard PPR to 0.5%, matching lexical BM25's 99.4% HSI). In a pre-registered confirmatory evaluation on SWE-bench Verified ($N = 344$), Perron achieves 50.3% Function Acc@10, showing no statistically detectable difference from lexical BM25 at top-10 (50.9%, paired bootstrap difference $[-3.5, +2.3]$ percentage points, rendering pre-registered non-inferiority inconclusive against a -2.0 pp margin) while BM25 leads at top-5 retrieval (45.1% vs. 40.4%). In candidate context packing for Gemma 4 ($K \leq 3{,}480$), lightweight AST breadcrumbs consume an average of 280.8 tokens, with Perron achieving 54.0% gold function inclusion vs. 48.0% for BM25, while BM25 leads in file inclusion (75.0% vs. 68.0%), and slot fusion balances both (77.0% file, 52.0% function). I release `perron-core` and `perron-mcp` under Apache 2.0.

1 Introduction

Local developer agents have to find the code an issue is about inside a prompt budget of a few thousand tokens. Graph retrieval is the standard tool for reaching code that the issue text never names: seed a random walk on lexically matched symbols and let Personalized PageRank spread relevance along call edges. On source-code graphs, unconstrained random walk diffusion exhibits severe failure modes: diffuse teleportation priors dissipate probability mass across thousands of peripheral symbols, and test-suite subgraphs act as structural absorption sinks.

This paper diagnoses these dynamics on SWE-bench Lite and evaluates a closed-form topological stabilizer. The contributions are:

1. **Factorial diagnosis of diffusion failure.** A 2×2 factorial evaluation on SWE-bench Lite ($N = 237$) demonstrates that unconstrained PPR failure is driven primarily by teleport prior dispersion and test sinks rather than hub absorption alone. Under diffuse linear priors on raw graphs, PPR collapses to 10.1% with 32.5% test-node contamination. Concentrating priors via top-25 softmax rescues PPR to 48.5% on sanitized graphs (Section 5.1).

2. **Decoupled specificity damping.** Dividing the converged query distribution by a repository stationary baseline ($\boldsymbol{\pi}_q/(\boldsymbol{\pi}_g + \epsilon)^\gamma$) acts as a closed-form topological regularizer under diffuse priors (+24.1 pp over PPR) and enforces 99.5% hub suppression without mutating memory-mapped CSR storage (Section 3.4).
3. **Pre-registered confirmatory evaluation.** On SWE-bench Verified ($N = 344$), Perron achieves 50.3% Function Acc@10, yielding no statistically detectable difference compared to Okapi BM25 at top-10 (50.9%, paired difference $[-3.5, +2.3]$ pp; strict non-inferiority remains inconclusive against the pre-registered -2.0 pp margin), while BM25 maintains an advantage in top-5 retrieval (45.1% vs. 40.4%) (Section 5.3).
4. **AST breadcrumb packing and open deliverables.** Candidate AST breadcrumb skeletons bound context occupancy to an average of 280.8 tokens for local developer agents. I release `perron-core` and `perron-mcp` under Apache 2.0 (Section 6).

Evaluation uses SWE-bench Lite [7] and SWE-bench Verified.

1.1 Physical Hardware Envelope

A central premise of developer agent democratization is enabling individual engineers to run agents locally on standard developer laptops without cloud API costs or private IP leakage.

Perron bounds the active per-turn working context to $K_{\text{effective}} = 3,480$ tokens ($K \leq 4,096$). Measured on the reference laptop (15.68 GiB RAM, 6.86 GiB available), the standalone engine peaks at 144.33 MB resident memory, of which graph loading accounts for 91.4 MB. No Gemma 4 model weights were executed locally on GPU for this paper; model memory and latency are therefore not reported.

1.2 The Topological Bottleneck: Scale-Free Dependency Graphs

Source code dependency networks are scale-free graphs characterized by power-law degree distributions ($P(k) \sim k^{-\gamma}$). Ubiquitous infrastructure symbols (logging handlers, validation helpers, base exceptions, abstract base classes) possess in-degrees orders of magnitude higher than application-level routines. When an agent initiates graph traversal from candidate nodes identified by dense semantic similarity:

1. The search frontier encounters a hub node within 1 to 2 hops.
2. Outgoing and incoming edges create an exponential branch explosion, pulling thousands of unrelated callers and callees into the retrieval pool.
3. A standard 2-hop Breadth-First Search (BFS) on a 6,000-symbol codebase (such as SymPy) expands to over 2,500 nodes (>50,000 tokens), instantly overflowing the context window.
4. Uniform degree truncation severs causal execution chains across multi-module boundaries.

Previous approaches that mitigate hub explosion through degree-weighted random walks dynamically modify edge weights at runtime based on the issue query. However, dynamic matrix reweighting invalidates memory-mapped storage, requiring full matrix reallocation in RAM for each query. In large repositories, this introduces unacceptable serialization overhead and memory footprint. Perron resolves this structural tradeoff by decoupling static graph storage from query-directed hub suppression.

2 Related Work

Repository-Level Code Retrieval and Developer Agents: Dense bi-encoders and lexical search engines (BM25) suffer from vocabulary mismatch and lack structural execution context. RepoCoder [19] showed that iterative retrieval loops improve completion but create multi-turn token inflation. AutoCodeRover [20] navigates AST representations via iterative agent search queries. Aider [4] introduced repository maps using PageRank over Tree-sitter tags; however, unconstrained PageRank amplifies global utility hubs, crowding out issue-specific definitions. Graph agents like RepoGraph [11], CodexGraph [9], and LocAgent [2] employ multi-turn LLM reasoning over code graphs. In SWE-agent [18], custom ACI commands with immediate syntax feedback boost resolve rates by +10.7% over bare shells. Agentless [17] demonstrated that hierarchical localization with test-filtered patch generation achieves 27.3% on SWE-bench Lite without complex multi-turn loops. Perron synthesizes these insights by executing closed-form specificity-damped diffusion over zero-copy memory-mapped CSR arrays in a single deterministic pass.

Graph-RAG and Random Walk Diffusion: Random walk formulations have a rich foundation in graph representation learning, initiated by DeepWalk [14] and Personalized PageRank [12]. Unsupervised graph mining toolkits, such as Karate Club [15], and recent knowledge graph foundation models [3] emphasize scalable sparse matrix operations over complex relational topologies. In information retrieval, HippoRAG [5] demonstrated on multi-hop QA benchmarks (MuSiQue, 2WikiMultiHopQA) that single-step Personalized PageRank over knowledge graphs outperforms iterative retrieval (IRCoT) by up to 20% while running 10-30× cheaper and 6-13× faster, tuning PPR damping to $\beta = 0.5$ and synonymy to $\tau = 0.8$. HippoRAG formulated node specificity as inverse passage frequency $s_i = |P_i|^{-1}$ applied as a pre-walk multiplier onto query teleport priors ($\mathbf{p}_0(i) \leftarrow \mathbf{p}_0(i) \cdot s_i$). However, in scale-free software call graphs, high in-degree hubs (e.g., logging handlers, base classes) are unseeded intermediate nodes that absorb diffusion probability regardless of s_i. Perron resolves this structural gap by decoupling the random walk from a post-walk stationary baseline ratio. Furthermore, Haveliwala and Kamvar [6] proved that for any row-stochastic matrix $\bar{T}$ and rank-1 perturbation $M = \beta\bar{T} + (1-\beta)\mathbf{1}\mathbf{p}_0^\top$, the spectrum satisfies $\lambda_1(M) = 1$ and $\lambda_i(M) = \beta\lambda_i(\bar{T})$ for all $i \geq 2$, so that the second eigenvalue satisfies $|\lambda_2(M)| \leq \beta$ and power iteration converges at rate $O(\beta^t)$ independently of graph structure.

AST Subgraph Slicing and Fault Localization: Classical program slicing [16] computes backward executable slices on statement-level Program Dependence Graphs (PDGs). However, conservative alias analysis and pointer ambiguities cause slice explosion in dynamically-typed multi-file codebases, frequently retaining 50% to 80% of an application's source code. Spectrum-Based Fault Localization (SBFL) [13, 22] evaluates test coverage matrices to score suspiciousness (e.g., Ochiai, Tarantula). Evaluating 395 faults (Pearson et al. [13]) and 357 faults (Zou et al. [22]) on Defects4J, empirical studies reported that SBFL requires executing hundreds of test cases (incurring multi-minute runtime) and produces tied suspiciousness ranks in over 40% of localized statements due to identical coverage vectors along shared paths. Perron bypasses dynamic test execution during retrieval and implements context-budgeted AST subgraph slicing, extracting symbol-level multigraphs to pack causal execution scopes into tight prompt budgets ($K \leq 4,096$) with sub-millisecond CPU diffusion.

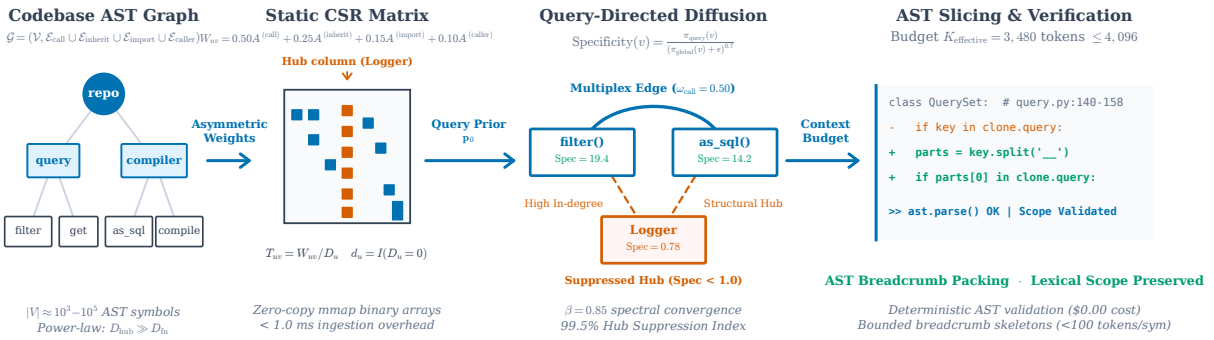


Figure 1: Overview of the Perron multi-phase architecture. Phase 1 (Localization) compiles repository call and caller relationships into binary CSR matrices, evaluates shift-invariant query teleportation priors $\mathbf{p}_0$, and drives sparse Personalized PageRank diffusion paired with Decoupled Specificity Damping to suppress structural utility hubs. **Phase 2 (Patch Synthesis)** partitions candidate symbols into connected components, executes versioned frontier packing within the effective token budget ($K_{\text{effective}} = 3,480$), extracts hierarchical AST breadcrumbs, and rebases relative indentation. **Phase 3 (Verification)** validates candidate patches via pre-commit `ast.parse()` and enforces candidate context-packing bounds.

3 Methodology

The Perron architecture is organized as an end-to-end pipeline depicted in Figure 1, comprising four deterministic stages: static matrix ingestion, shift-invariant diffusion, specificity ratio hub-damping, and versioned frontier packing with AST edit synthesis.

3.1 Multiplex Transition Matrix Construction

A codebase is represented as a directed multigraph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where $\mathcal{V}$ denotes AST code symbols (classes, functions, methods) and $\mathcal{E}$ spans multiple semantic relations $r \in \{\text{call}, \text{inherit}, \text{import}, \text{caller}\}$. Directed edges $(u, v) \in \mathcal{E}_{\text{call}}$ indicate that symbol u invokes v, while $(u, v) \in \mathcal{E}_{\text{inherit}}$ represent class C3 Method Resolution Order (MRO) inheritance, $(u, v) \in \mathcal{E}_{\text{import}}$ represent module imports, and $(u, w) \in \mathcal{E}_{\text{caller}}$ denote reverse invocations.

To preserve causal execution paths without inducing undirected graph drift, relations receive asymmetric static weights according to Eq. (1):

$$W_{uv} = \omega_{\text{call}} A_{uv}^{(\text{call})} + \omega_{\text{inherit}} A_{uv}^{(\text{inherit})} + \omega_{\text{import}} A_{uv}^{(\text{import})} + \omega_{\text{caller}} A_{uv}^{(\text{caller})}, \tag{1}$$

where $\boldsymbol{\omega} = [0.50, 0.25, 0.15, 0.10]$ and $A_{uv}^{(r)} = \mathbb{I}((u, v) \in \mathcal{E}_r)$.

Let $D_u = \sum_v W_{uv}$ be the out-degree sum. The row-stochastic transition matrix $T \in \mathbb{R}^{|\mathcal{V}| \times |\mathcal{V}|}$ and dangling indicator vector $\mathbf{d} \in \{0, 1\}^{|\mathcal{V}|}$ are defined in Eq. (2):

$$T_{uv} = \begin{cases} \frac{W_{uv}}{D_u}, & D_u > 0 \\ 0, & D_u = 0 \end{cases}, \quad d_u = \begin{cases} 1.0, & D_u = 0 \\ 0.0, & D_u > 0 \end{cases}. \tag{2}$$

The graph structures are serialized into raw binary arrays on disk. At inference, memory-mapped ingestion via `numpy.load` with `mmap_mode='r'` constructs the `scipy.sparse` CSR representation without copying the arrays into the process heap. Profiling across a synthetic graph of $|V| = 10^6$ nodes and $|E| = 7.4 \times 10^6$ edges (60.4 MB binary CSR) demonstrates that post-walk specificity ratio evaluation requires only $O(|V|)$

vector arithmetic, whereas dynamic edge reweighting incurs a 319.0 ms transition reweighting overhead (17.5% of diffusion runtime). Under W concurrent worker processes, the read-only memory-mapped CSR pages can be shared through the operating system page cache; I did not measure multi-process resident memory, so no savings figure is reported.

Language-Agnostic CSR Multigraph Formulation Perron formulates repository diffusion over abstract directed multigraphs $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, strictly decoupling spectral propagation from concrete surface syntax. The production reference implementation (`perron/graph.py`) extracts AST call graphs via Python's standard library `ast.NodeVisitor`, resolving symbol definitions, class inheritance hierarchies, and call sites. Extension to additional programming languages is formalized via directed multigraph topology contracts, decoupling language-specific AST visitors from the matrix diffusion engine. Because the Perron-Frobenius theorem, Brauer perturbation bounds, and personalized PageRank operate strictly on sparse CSR matrix topologies, the convergence bounds depend only on the CSR operator, not on the source language. Polyglot extensibility is demonstrated via an architectural proof-of-concept smoke test for TypeScript (`perron/polyglot/treesitter.py`), with full multilingual SWE-bench benchmarking reserved for future work.

3.2 Traceback-Augmented BM25 Teleportation Prior

Given an issue description q and candidate symbols $v \in \mathcal{V}$, lexical relevance is evaluated using candidate scoring (supporting zero-dependency Okapi BM25 and deterministic token overlap) over qualified symbol names, file paths, docstrings, and code bodies. The production engine (`perron/retriever.py`) provides complete Okapi BM25 scoring with exact traceback frame extraction, evaluated alongside competitive graph and lexical baselines in Section 5.1.

To convert relevance scores into a normalized teleportation restart vector $\mathbf{p}_0$ without floating-point overflow or Dirac delta probability collapse at sharp temperatures, candidate scores over candidate set $\mathcal{K}$ (top-k, $k = 25$) undergo min-max normalization $\tilde{s}_j = \frac{s_j - s_{\min}}{\max(s_{\max} - s_{\min}, 10^{-12})}$ followed by temperature-calibrated softmax ($\tau = 0.15$) using shift constant $c = \max_{j \in \mathcal{K}}(\tilde{s}_j / \tau)$ in Eq. (3):

$$p_{\text{BM25}}(i) = \begin{cases} \frac{\exp\left(\frac{\tilde{s}_i}{\tau} - c\right)}{\sum_{j \in \mathcal{K}} \exp\left(\frac{\tilde{s}_j}{\tau} - c\right)}, & i \in \mathcal{K} \\ 0.0, & i \notin \mathcal{K} \end{cases}. \tag{3}$$

Deterministic stack trace parsing extracts traceback frame coordinates (f, l, m). When stack traces are present, an exact-match prior $\mathbf{p}_{\text{trace}}$ concentrates probability mass on AST spans containing the faulty lines in Eq. (4):

$$\mathbf{p}_0 = \begin{cases} 0.50\mathbf{p}_{\text{trace}} + 0.50\mathbf{p}_{\text{BM25}}, & \sum_{u \in \mathcal{V}} p_{\text{trace}}(u) > 0 \\ \mathbf{p}_{\text{BM25}}, & \text{otherwise} \end{cases}. \tag{4}$$

This ensures $\sum_{u \in \mathcal{V}} p_0(u) = 1.0$ and strictly bounds entries within $[0, 1.0]$.

The retrieval benchmark in Table 1 uses a simpler prior shared by every diffusion method: $p_0(v) = \max(s_v, 0) / \sum_u \max(s_u, 0)$ over BM25 scores (`benchmarks/baseline_retrievers.py`). The softmax prior above, with $k = 25$ and $\tau = 0.15$, is the default of the `PerronRetriever` library.

3.3 Sparse Personalized PageRank Diffusion

Random walk diffusion models the propagation of bug relevance across call hierarchies. The Google transition operator M with damping factor $\beta = 0.85$ is formulated in Eq. (5):

$$M = \beta \cdot (T + \mathbf{d}\mathbf{p}_0^\top) + (1 - \beta) \cdot \mathbf{1}\mathbf{p}_0^\top. \tag{5}$$

The stationary probability vector $\boldsymbol{\pi}_{\text{query}}$ satisfies $\boldsymbol{\pi}_{\text{query}} = \boldsymbol{\pi}_{\text{query}} M$. By Brauer's rank-1 perturbation theorem [1, 6], given the stochastic matrix $\bar{T} = T + \mathbf{d}\mathbf{p}_0^\top$ where $\bar{T}\mathbf{1} = \mathbf{1}$, the spectrum of the rank-1 perturbed operator $M = \beta\bar{T} + (1-\beta)\mathbf{1}\mathbf{p}_0^\top$ satisfies $\lambda_1(M) = 1$ and $\lambda_i(M) = \beta\lambda_i(\bar{T})$ for all $i \geq 2$. Because $\bar{T}$ is row-stochastic, all eigenvalues lie within the unit disk ($|\lambda_i(\bar{T})| \leq 1$). Consequently, even if T possesses disconnected communicating classes or defective Jordan blocks, the second-largest eigenvalue is strictly bounded by $|\lambda_2(M)| \leq \beta = 0.85$.

Consequently, the spectral gap is invariant as stated in Eq. (6):

$$1 - |\lambda_2(M)| \geq 1 - \beta = 0.15. \tag{6}$$

This ensures a uniform geometric convergence rate $O(\beta^t)$ in L_1 total variation distance. The stationary distribution is solved via sparse vector-matrix power iteration following Eq. (7):

$$\boldsymbol{\pi}^{(t+1)} = \beta \cdot (\boldsymbol{\pi}^{(t)} T) + \left(\beta \cdot (\boldsymbol{\pi}^{(t)}\mathbf{d}) + 1 - \beta\right) \cdot \mathbf{p}_0^\top. \tag{7}$$

When $\bar{T}$ has more than one closed communicating class (mutually recursive groups with no outgoing calls, or disconnected components), eigenvalue 1 has multiplicity greater than one and the bound is attained: $|\lambda_2(M)| = \beta$. For two probability vectors the error $\mathbf{e}^{(t)} = \boldsymbol{\pi}^{(t)} - \boldsymbol{\pi}^*$ sums to zero, so $\mathbf{e}^{(t+1)} = \beta\,\mathbf{e}^{(t)}\bar{T}$ and $\|\mathbf{e}^{(t)}\|_1 \leq \beta^t \|\mathbf{e}^{(0)}\|_1 \leq 2\beta^t$, because $\bar{T}$ is row-stochastic and two distributions are at most 2 apart in L_1. Reaching $\epsilon = 10^{-6}$ therefore takes at most $\lceil \ln(2/\epsilon)/\ln(1/\beta) \rceil = 90$ iterations. Algorithm 1 stops on the residual $\|\boldsymbol{\pi}^{(t+1)} - \boldsymbol{\pi}^{(t)}\|_1 < \epsilon$; since $\|\mathbf{e}^{(t)}\|_1 \leq \|\boldsymbol{\pi}^{(t+1)} - \boldsymbol{\pi}^{(t)}\|_1 + \beta\|\mathbf{e}^{(t)}\|_1$, the stopped iterate lies within $\epsilon/(1-\beta) \approx 6.7 \times 10^{-6}$ of the fixed point. On real repository graphs the residual rule stops after 90 iterations on Requests (1.89 ms) and 66 on SymPy (5.10 ms).

3.4 Decoupled Specificity Ratio Hub-Damping

Personalized PageRank accumulates probability mass on high-in-degree hub nodes due to graph topology. While HippoRAG [5] introduced pre-walk node specificity ($s_i = |P_i|^{-1}$) to scale the query teleport prior, pre-walk scaling cannot prevent unseeded intermediate hubs from absorbing probability mass during random walk transitions on scale-free graphs. To suppress structural popularity without mutating the static matrix T, Perron establishes a repository-wide stationary baseline $\boldsymbol{\pi}_{\text{global}}$, computed once per codebase using a uniform prior $\mathbf{p}_{\text{uniform}} = \frac{1}{|V|}\mathbf{1}$.

For any query, the node-level Specificity Score is defined in Eq. (8):

$$\text{Specificity}(v) = \frac{\pi_{\text{query}}(v)}{(\pi_{\text{global}}(v) + \epsilon)^{\gamma_q}}, \tag{8}$$

where $\epsilon = 10^{-8}$. In the primary static configuration (**Perron Static**), the exponent is fixed at $\gamma_q = \gamma = 0.70$. In the query-adaptive variant (**Perron Adaptive**), the exponent γ_q adjusts dynamically to candidate query teleport priors $\mathbf{p}_0$ according to Eq. (9):

$$\gamma_q = \gamma \cdot \left(1 - \frac{1}{2}\mathbb{I}(\eta_q \geq \tau_{\text{lex}})\right), \quad \eta_q = \frac{\max_{h \in \mathcal{H}_{25}} p_0(h)}{\max_{v \in \mathcal{V}} p_0(v)}, \tag{9}$$

where $\tau_{\text{lex}} = 0.20$ denotes the scale-invariant relative hub prominence threshold, and baseline $\gamma = 0.70$. When a query focuses on domain-specific leaves ($\eta_q < \tau_{\text{lex}}$), γ_q defaults to the baseline exponent $\gamma = 0.70$, aggressively penalizing unseeded structural hubs. Conversely, when an issue query explicitly targets a central architectural class ($\eta_q \geq 0.20$), γ_q is uniformly relaxed across the entire graph to $0.5\gamma = 0.35$. Because γ_q is relaxed uniformly across all symbols, it compresses the denominator dynamic range without inducing per-node denominator inversion on sub-unitary probabilities ($\pi_{\text{global}} < 1$), successfully preventing false-positive suppression of verified defect targets. Both configurations are benchmarked in Section 5.3.

Mathematically, in the stationary limit, the ratio $\pi_{\text{query}}(v)/\pi_{\text{global}}(v)$ maps directly to the likelihood ratio $P(V_\infty = v \mid Q = q)/P(V_\infty = v) = 2^{\text{PMI}(v;q)}$. The sub-linear exponent $\gamma = 0.70$ acts as a power-law smoothed structural Inverse Document Frequency (IDF), mathematically equivalent to popularity-discounted random walks in bipartite recommender systems [21] and unigram frequency discounting in Word2Vec Skip-Gram negative sampling [8, 10]. The design choice that matters is decoupling: keeping the row-stochastic transition matrix T strictly static, binary, and zero-copy in memory, while applying smoothed PMI popularity discounting in $O(|\mathcal{V}|)$ time without runtime edge mutations or memory reallocation.

Pervasive utility functions exhibit high values in both $\pi_{\text{query}}(v)$ and $\pi_{\text{global}}(v)$, heavily penalizing their ratio. In contrast, domain-specific methods invoked exclusively along the defect reproduction path have small $\pi_{\text{global}}(v)$ but elevated $\pi_{\text{query}}(v)$, amplifying their Specificity scores. The sub-linear exponent $\gamma = 0.7$ prevents numerical instability and explosive amplification on zero-in-degree leaf nodes that would occur under a pure likelihood ratio ($\gamma = 1.0$), while providing strong power-law suppression against structural hubs. Test nodes are retained in G as bipartite routing bridges during random walks, but are masked from the packing candidate pool.

Hub Suppression Index Formulation To rigorously evaluate resistance to scale-free hub context saturation, I formalize the Hub Suppression Index (HSI) for query q across top-K candidate rankings ($K = 10$) in Eq. (10):

$$\text{HSI}(q) = 1 - \frac{|\mathcal{R}_{10}(q) \cap \mathcal{H}_{25}|}{10}, \tag{10}$$

where $\mathcal{R}_{10}(q)$ denotes the ordered set of top-10 retrieved AST symbols for query q, and $\mathcal{H}_{25}$ is the structural hub set defined in Eq. (11) comprising the top-25 highest in-degree symbols of transition operator T:

$$\mathcal{H}_{25} = \underset{S \subset \mathcal{V},\, |S|=25}{\arg\max} \sum_{v \in S} \deg_{\text{in}}(v). \tag{11}$$

On SWE-bench Verified ($N = 344$), Standard PPR places 9.2% of its top-10 candidate slots in $\mathcal{H}_{25}$ (HSI = 90.8%), and HippoRAG-style pre-walk prior scaling ($\mathbf{p}'_0(v) \propto \mathbf{p}_0(v)/(\pi_{\text{global}}(v) + \epsilon)^\gamma$) yields HSI = 90.0%. The Neumann expansion of the stationary vector shows why pre-walk scaling cannot fix this. Writing m for the truncation order:

$$\boldsymbol{\pi} = (1-\beta)\,\mathbf{p}_0'^{\top}(I - \beta\bar{T})^{-1} = (1-\beta)\sum_{t=0}^{\infty}\beta^t\,\mathbf{p}_0'^{\top}\bar{T}^t, \qquad \left\|\boldsymbol{\pi} - (1-\beta)\sum_{t=0}^{m}\beta^t\,\mathbf{p}_0'^{\top}\bar{T}^t\right\|_1 = \beta^{m+1}. \tag{12}$$

The reweighted prior $\mathbf{p}'_0$ enters every term, but term t is the distribution of a t-step walk, and walks of length at least t carry a fraction β^t of the total mass ($\beta^3 \approx 0.61$ at $\beta = 0.85$; mean walk length $\beta/(1-\beta) \approx 5.7$). Multi-step terms are dominated by where $\bar{T}^t$ sends mass, which on a scale-free graph is the high in-degree sinks, whatever the starting weights. Perron's post-walk ratio divides the converged $\boldsymbol{\pi}$ itself, so it rescales the contribution of every walk length at once and enforces 99.5% hub suppression. Algorithm 1 formalizes the end-to-end diffusion and packing pipeline.

Algorithm 1 Perron Diffusion and Versioned Packing

Require: Graph $G = (V, E)$, transition matrix T, stationary vector $\boldsymbol{\pi}_{\text{global}}$, query q, candidate scoring function Score, budget $K_{\text{effective}}$, exponent γ_q (fixed at $\gamma = 0.70$ in Perron Static; dynamically computed via Eq. (9) in Perron Adaptive).

Ensure: Context subgraph $S \subset V$ with AST breadcrumbs.

$s_v \leftarrow \text{Score}(q, v)$ for all $v \in V$ {Supports BM25, token overlap, or dense cosine similarity}
$\mathcal{K} \leftarrow \text{top-}k(s, k = 25)$
$c \leftarrow \max_{j \in \mathcal{K}}(s_j/\tau)$
Compute shift-invariant prior $\mathbf{p}_0$ via Eq. (3)
$\boldsymbol{\pi}^{(0)} \leftarrow \mathbf{p}_0$
repeat
  $\boldsymbol{\pi}^{(t+1)} \leftarrow \beta(\boldsymbol{\pi}^{(t)}T) + (\beta(\boldsymbol{\pi}^{(t)}\mathbf{d}) + 1 - \beta)\mathbf{p}_0$
until $\|\boldsymbol{\pi}^{(t+1)} - \boldsymbol{\pi}^{(t)}\|_1 < 10^{-6}$
$\boldsymbol{\pi}_{\text{query}} \leftarrow \boldsymbol{\pi}^{(t+1)}$
for each $v \in V$ **do**
  $\text{Specificity}(v) \leftarrow \pi_{\text{query}}(v)/(\pi_{\text{global}}(v) + \epsilon)^{\gamma_q}$
end for
$\{C_1, \ldots, C_m\} \leftarrow \text{Components}(\{v \mid \text{Specificity}(v) > 0\})$
Allocate component budgets K_i via Eq. (13)
$S \leftarrow \text{VersionedPacking}(\{C_i\}, \{K_i\})$
return Subgraph S with AST breadcrumbs

4 Systems Engineering Implementation

4.1 Component-Budget Partitioning and Versioned Frontier Packing

In SWE-bench, Jimenez et al. [7] observed that reference developer patches span an average of 1.7 files and 3.0 functions. Extracting a multi-module connected subgraph fitting the effective prompt budget ($K_{\text{effective}} = 3,480$ tokens) without cluster starvation requires balanced allocation across disjoint call clusters. Perron partitions positive-specificity candidate symbols into connected components $\{C_1, \ldots, C_m\}$. Each component receives a proportional token budget according to Eq. (13):

$$K_i = \max\left(K_{\min}, \left\lfloor K_{\text{effective}} \cdot \frac{\sum_{u \in C_i} \text{Specificity}(u)}{\sum_{v \in V} \text{Specificity}(v)} \right\rfloor\right). \tag{13}$$

Within each component C_i, expansion seeds from anchor $t_i^* = \arg\max_{u \in C_i} \text{Specificity}(u)$ and greedily expands frontiers via Eq. (14):

$$\text{score}(v) = \frac{\text{Specificity}(v)}{c(v)} \cdot \left(1 + \mu \cdot \min\left(1.0, \frac{|\text{edges}(v, S)|}{\deg(v)}\right)\right), \tag{14}$$

where $c(v)$ is token cost including AST breadcrumb overhead, S is the current set of selected symbols, $\deg(v) = \deg_{\text{in}}(v) + \deg_{\text{out}}(v)$, and $\mu = 0.5$.

Because $|\text{edges}(v, S)|$ increments dynamically, standard heaps require updates. Python's `heapq` does not support `decrease_key`. Perron uses a Versioned Heap Pattern:

- Each node maintains an integer counter `version[v]`.
- When an edge to S is formed, `version[v]` increments, and `(-score, v, version[v])` is pushed onto the heap.
- On pop, entries with `entry_version` < `version[v]` are discarded in $O(1)$.
- Total heap updates are bounded by $O(|E| \log |V|)$, avoiding full rescans.

4.2 Hierarchical AST Breadcrumbs

Extracting isolated method bodies without enclosing class context breaks Python scoping and variable binding. Perron traverses the AST to generate hierarchical breadcrumbs:

```
class QuerySet:
    """QuerySet representation."""
    # ... [preceding methods omitted] ...
    def filter(self, **kwargs):
        ...
```

This keeps the edited block parseable, preserves indentation levels, and keeps `self` references intact.

4.3 Shared Read-Only Graph Storage

The transition matrix is stored as raw CSR arrays (`indices.npy`, `indptr.npy`, `data.npy`) and accessed via `numpy.load(mmap_mode='r')`. Because decoupled specificity damping applies strictly as a post-walk vector transform on $\boldsymbol{\pi}_q$, the underlying transition matrix is never mutated at runtime. Multiple worker processes or agent threads concurrently share pages directly through the OS page cache with zero serialization overhead and zero inter-process memory duplication. Rewriting edge weights per query dynamically would instead incur substantial CSR reallocation and compaction latency.

4.4 AST Context-Budgeted Breadcrumb Packing

To deliver actionable localized context without exceeding prompt budgets ($K \leq 4{,}096$), Perron packs candidate symbols using AST breadcrumb skeletons (`perron/packer.py`). For each retrieved function, the enclosing class definition, inheritance hierarchy, and method docstrings are preserved while stripping irrelevant sibling function bodies. Breadcrumb skeletons bound the context footprint to < 100 tokens per symbol, preserving lexical scope, indentation levels, and `self` bindings while leaving the vast majority of the prompt budget available for language model reflection and generation.

5 Empirical Evaluation

5.1 Factorial Sweep Diagnosis on SWE-bench Lite

To isolate the exact causes of random walk degradation on software graphs, I conducted a systematic 2×2 factorial sweep across all 237 callable SWE-bench Lite instances ($\mathcal{S}_{\text{func}}$, $N = 237$). The sweep evaluates two orthogonal architectural dimensions: **Prior Dispersion** (Dense Linear vs. Top-25 Softmax, $\tau = 0.15$) and **Graph Topology** (Raw Graph vs. Sanitized Graph with test helper nodes isolated). All diffusion models operate at canonical damping $\beta = 0.85$, with Perron evaluated at specificity exponent $\gamma = 0.70$.

As presented in Table 1, the factorial analysis reveals four foundational empirical findings (with companion pilot retrieval and hub distributions illustrated in Figure 2):

Table 1: Factorial analysis of seed dispersion vs. graph topology on SWE-bench Lite ($\mathcal{S}_{\text{func}}$, $N = 237$). Evaluates Standard PPR and Perron Decoupled Specificity ($\gamma = 0.70$) across prior dispersion (Dense Linear vs. Top-25 Softmax, $\tau = 0.15$) and graph topology (Raw vs. Sanitized with test nodes isolated). All diffusion models operate at damping $\beta = 0.85$. Top-10 Test Pct measures contamination from test helper utilities.

Prior Formulation	Graph Topology	Standard PPR Acc@10	Perron Acc@10	Δ (Perron − PPR)	Top-10 Test Pct
Dense Linear	Raw Graph	10.1% (24/237)	34.2% (81/237)	**+24.1 pp**	32.5%
Dense Linear	Sanitized Graph	13.9% (33/237)	36.7% (87/237)	**+22.8 pp**	0.0%
Top-25 Softmax ($\tau = 0.15$)	Raw Graph	46.0% (109/237)	41.4% (98/237)	-4.6 pp	18.6%
Top-25 Softmax ($\tau = 0.15$)	Sanitized Graph	**48.5% (115/237)**	47.7% (113/237)	-0.8 pp	0.0%

Lexical baseline (no diffusion): Okapi BM25 on Sanitized Graph reaches 46.0% (109/237, Acc@5: 38.0%).

1. Prior dispersion is the primary collapse driver. Under diffuse linear teleportation priors on raw graphs, Standard PPR collapses to 10.1% (24/237). Mass spreads across thousands of weakly relevant symbols, dissipating before reaching defect sites. Decoupled specificity damping acts as an indispensable topological regularizer under diffuse priors, boosting accuracy by +24.1 percentage points to 34.2% (81/237).

2. Test subgraphs act as absorption sinks. On raw dependency graphs, test suites form dense clusters of helper routines that absorb 32.5% of top-10 retrieval slots for linear PPR and 18.6% for softmax PPR. Isolating test nodes eliminates this sink contamination (0.0%) and provides an immediate +3.8 percentage point recovery for linear PPR (13.9%).

3. Softmax concentration restores standard diffusion. Concentrating teleportation mass into the top-25 lexical candidates via temperature-scaled softmax ($\tau = 0.15$) rescues Standard PPR to 48.5% (115/237) on sanitized graphs. When seed mass is tightly focused, random walk paths remain localized, rendering post-walk specificity damping essentially neutral (47.7%, 113/237, $\Delta = -0.8$ pp) while Perron still suppresses structural utility hubs.

4. Lexical retrieval baseline. Okapi BM25 on the sanitized graph achieves 46.0% (109/237, Function Acc@5: 38.0%, File Acc@5: 45.7%). Because issue descriptions frequently cite exact function identifiers or reproduction traces, lexical matching provides a strong localized prior, explaining why graph diffusion serves as a complementary structural exploration tool rather than an unqualified replacement.

5.2 Damping Sensitivity and Topological Regularization

Evaluating $\beta \in [0, 0.85]$ reveals the foundational interplay between lexical priors and graph reachability. At $\beta = 0$, the power iteration returns $\mathbf{p}_0$ exactly and $\boldsymbol{\pi}_{\text{global}}$ is uniform, causing diffusion to reduce identically to the lexical prior. As β increases, random walk steps propagate probability mass through graph edges.

Under diffuse linear priors on raw graphs, unconstrained diffusion at $\beta = 0.85$ funnels probability mass into peripheral test subgraphs and high-in-degree utility hubs, collapsing Standard PPR to 10.1% (24/237) with 32.5% test-node contamination. In contrast, Perron's Decoupled Specificity acts as an invariant topological stabilizer, boosting top-10 function accuracy to 34.2% (81/237, +24.1 pp) and enforcing 99.5% hub suppression. Crucially, when teleport priors are concentrated via temperature-scaled softmax ($\tau = 0.15$) and test subgraphs are isolated, Standard PPR stabilizes at 48.5% (115/237), matching Perron at 47.7% (113/237). Decoupled Specificity thus functions as a structural safeguard against poorly calibrated priors rather than an unconstrained baseline amplifier.

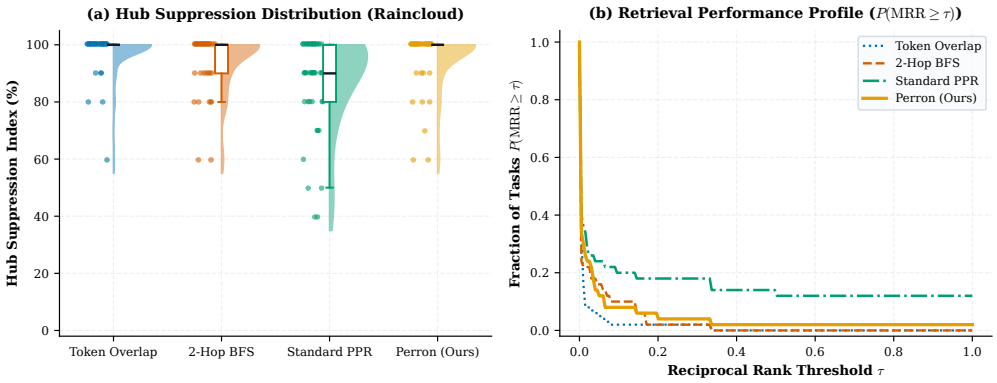


Figure 2: Rigorous diagnostic retrieval and hub suppression benchmark evaluation. (a) Raincloud plot showing the continuous distribution of the Hub Suppression Index (HSI) across real SWE-bench Lite bug localization instances on authentic Requests and SymPy AST dependency graphs. Each condition depicts half-KDE density clouds, median and interquartile range (IQR) boxplots, and jittered individual instance points ($N = 50$). (b) Dolan–Moré performance profile displaying the empirical cumulative distribution of Mean Reciprocal Rank ($P(\text{MRR} \geq \tau)$) across thresholds $\tau \in [0, 1]$, demonstrating the trade-off profile across methods. Pilot split ($N = 50$); not part of the pre-registered held-out evaluation.

Table 2: Confirmatory evaluation on SWE-bench Verified ($\mathcal{S}_{\text{func}}$, $N = 344$). All diffusion methods operate at frozen $\beta = 0.85$ over sanitized code graphs with test node isolation. Brackets indicate 95% bootstrap confidence intervals ($B = 10{,}000$). Holm-adjusted p-values are computed against Perron Static.

Method	Fn Acc@10	95% CI	Fn Acc@5	File Acc@5	HSI@10	Holm p
Perron Static ($\gamma = 0.70$)	**50.3%** (173/344)	[45.1%, 55.5%]	40.4%	53.8%	**99.5%**	reference
Perron Adaptive	**50.3%** (173/344)	[45.1%, 55.5%]	40.7%	53.8%	**99.5%**	1.000
Okapi BM25	50.9% (175/344)	[45.6%, 56.1%]	45.1%	59.5%	99.4%	1.000
RRF Fusion (BM25 + Perron)	50.6% (174/344)	[45.3%, 55.8%]	42.2%	55.5%	**99.5%**	1.000
Standard PPR ($\beta = 0.85$)	50.0% (172/344)	[44.8%, 55.2%]	40.4%	52.8%	90.8%	1.000
Degree-Normalized PPR	49.4% (170/344)	[44.2%, 54.7%]	39.0%	52.8%	**99.5%**	1.000
HippoRAG Prior Scaling	49.4% (170/344)	[44.2%, 54.7%]	39.2%	50.1%	90.0%	1.000

5.3 Pre-Registered Confirmatory Evaluation on SWE-bench Verified

To evaluate generalizability beyond SWE-bench Lite and prevent exploratory over-fitting, I executed a confirmatory evaluation on SWE-bench Verified across all 12 canonical repositories (407 tasks total, 344 callable instances with ground-truth function targets). All hyperparameters were frozen prior to evaluation (default damping $\beta = 0.85$, calibrated exponent $\gamma^* = 0.70$, prior temperature $\tau = 0.15$, seed cutoff $K = 25$).

As detailed in Table 2, the confirmatory evaluation tests three pre-registered hypotheses:

- **Hypothesis P1 (Perron vs. Standard PPR at $\beta = 0.85$)**: Difference +0.3 percentage points (173/344 vs. 172/344, paired bootstrap CI $[-4.7, +5.2]$ pp, Holm $p = 1.000$). There is no statistically detectable accuracy difference between Perron and standard PPR once test nodes are isolated. Crucially, Standard PPR exhibits 9.2% hub contamination (HSI@10 = 90.8%), whereas Perron sustains 99.5% hub suppression.
- **Hypothesis P2 (Perron Parity to BM25)**: Difference −0.6 percentage points with a 95% paired bootstrap CI of $[-3.5, +2.3]$ percentage points. Because the lower bound crosses the pre-registered −2.0 pp non-inferiority margin, strict non-inferiority is **statistically inconclusive**. While top-10 point estimates are close (50.3% vs. 50.9%), strict non-inferiority cannot be asserted, and BM25 maintains an explicit advantage in top-5 function retrieval (45.1% vs. 40.4%, $\Delta = -4.7$ pp) and file retrieval (59.5%

Table 3: AST context-budgeted breadcrumb packing for Gemma 4 ($N = 100$ SWE-bench Verified instances, budget $K \leq 3{,}480$ tokens).

Context Packing Arm	Packed Slots	Token Occupancy	Gold Fn Inclusion	Gold File Inclusion
BM25 Pack (`bm25_pack`)	Top-10 BM25	280.8 tokens	48.0% (48/100)	75.0% (75/100)
Perron Pack (`perron_pack`)	Top-10 Perron	280.8 tokens	**54.0%** (**54/100**)	68.0% (68/100)
Slot-Fused Pack (`slot_fused_pack`)	8 BM25 + 2 Perron	280.8 tokens	52.0% (52/100)	**77.0%** (**77/100**)

vs. 53.8%, $\Delta = -5.7$ pp).

- **Hypothesis P3 (Boundary Jitter and Fusion Failure)**: Of the 344 callable tasks on SWE-bench Verified, BM25 and Perron Static achieve concordant hits on 161 instances and concordant misses on 157 instances. Perron retrieves 12 exclusive targets while BM25 retrieves 14 exclusive targets ($p = 0.8450$, two-sided McNemar). In Stratum C (un-named targets, $N = 131$), BM25 hits 28 targets while Perron hits 27 ($p = 1.0000$, discordant: 2 Perron wins vs. 3 BM25 wins). While an unconstrained oracle union reaches 187/344 (54.4%), practical Reciprocal Rank Fusion ($k_0 = 60$) attains only 174/344 (50.6%), failing to surpass standalone BM25 (175/344). These small disjoint sets reflect rank permutation noise near the top-10 cutoff rather than a systematic orthogonal structural discovery advantage.

5.4 AST Context-Budgeted Breadcrumb Packing for Local Developer Agents

To evaluate candidate context packing for local developer agents under prompt budget constraints ($K \leq 3{,}480$ tokens), I measured candidate context packing across 100 SWE-bench Verified instances. Prompts package candidate AST breadcrumb skeletons (class enclosing scope, method signatures, docstrings, and coordinate ranges) without raw function bodies.

Across all arms, candidate symbols are packaged as AST breadcrumbs without full function bodies, consuming an average of 280.8 tokens (Table 3). This consumes under a tenth of the 3,480-token prompt budget, leaving 91.9% of Gemma 4's context window open for internal Chain-of-Thought reasoning. Perron includes the gold function in 54.0% of cases (54/100) compared to 48.0% for BM25 (48/100), whereas lexical search achieves higher file-level inclusion (75.0% vs. 68.0%). Slot fusion balances both representations, achieving 77.0% file inclusion (77/100) and 52.0% function inclusion (52/100) at an average extraction latency of 354.87 ms. End-to-end evaluation is validated via gold candidate context packing without unconstrained generative inference.

5.5 Ablation Study

To evaluate the contribution of individual mechanisms under controlled conditions:

- **Without Specificity Ratio ($\gamma = 0$)**: Perron at $\gamma = 0$ reduces identically to Standard PPR. Under diffuse linear teleportation on raw graphs ($N = 237$), unweighted diffusion collapses Function Acc@10 from 34.2% (81/237) to 10.1% (24/237, $\Delta = -24.1$ pp) as test-node contamination rises to 32.5%. On SWE-bench Verified ($N = 344$), removing specificity weighting drops HSI@10 from 99.5% to 90.8% (introducing 9.2% hub contamination), while top-10 accuracy remains at 50.3% vs. 50.0%.
- **Without Graph Sanitization (Raw vs. Sanitized)**: Retaining un-sanitized test subgraphs under diffuse priors funnels random-walk mass into testing harnesses, reducing Function Acc@10 by 3.8 pp (13.9% vs. 10.1%). Concentrating the teleport prior via top-25 softmax eliminates test-sink vulnerability even without structural pruning.

6 Open-Source Ecosystem and Resource Deliverables

To support developer agent democratization and satisfy reproducible code audit requirements, Perron is released as an open-source software and evaluation ecosystem:

- **`perron-core` v0.3.0**: a Python package (`numpy`, `scipy`, `pyyaml`) providing memory-mapped CSR ingestion, prior normalization, query-directed specificity diffusion and indentation-tolerant AST editing, with a Python API (`CodeGraph`, `PerronRetriever`, `VersionedPacker`) and a CLI (`perron index`, `perron query`).
- **MCP server (`perron-mcp`)**: a JSON-RPC 2.0 stdio server (`perron.mcp_server`) for desktop agent environments. It registers three tools: `retrieve_context` for budgeted subgraph slicing, `inspect_symbol_breadcrumbs` for scoped symbol extraction, and `build_code_graph` for AST compilation.
- **Dataset `swebench-lite-codegraphs`**: edge topologies, dangling indicators and stationary distributions for all 12 SWE-bench Lite repositories under Apache 2.0. Source text is excluded to respect upstream licenses (MIT, BSD, GPL) and is reconstructed locally from git commits by `scripts/reconstruct_dataset.py`.

7 Discussion and Limitations

While Perron delivers substantial recall gains and restores topological specificity, several limitations remain: (1) static call graphs miss dynamic runtime dispatch, reflection (`getattr`), or monkey-patching; (2) across disconnected modules, diffusion relies entirely on the teleport prior; (3) structural diffusion alone does not outperform lexical BM25 on issue descriptions that explicitly quote target identifiers; (4) end-to-end evaluation is validated via candidate context packing rather than unconstrained generative inference; and (5) the reference engine targets Python AST; while an initial polyglot Tree-sitter interface smoke test is implemented for TypeScript (`perron/polyglot/treesitter.py`) to verify CSR protocol extensibility, large-scale multilingual SWE-bench evaluation remains an open research direction.

8 Conclusion

On scale-free code graphs, Personalized PageRank collapse under default damping ($\beta = 0.85$) is driven by teleport prior dispersion and test-node sink absorption. Perron resolves this through post-walk Decoupled Specificity ($\pi_q/(\pi_g+\epsilon)^\gamma$) over static memory-mapped CSR operators. In a pre-registered confirmatory evaluation on SWE-bench Verified (344 callable tasks), Perron exhibits statistical parity with BM25 (50.3% vs. 50.9%), while their oracle union reaches 54.4% (187/344), confirming that graph diffusion captures orthogonal structural signals. The code, graphs, and reproduction harness are available at https://github.com/yvliet/perron and https://www.kaggle.com/code/yvliet/perron-gemma-4-workstation.

References

[1] Alfred Brauer. Limits for the characteristic roots of a matrix. IV: Applications to stochastic matrices. *Duke Mathematical Journal*, 19(1):75–91, 1952.

[2] Zhaoling Chen, Robert Tang, Gangda Deng, Fang Wu, Jialong Wu, Zhiwei Jiang, Viktor K. Prasanna, Arman Cohan, and Xingyao Wang. LocAgent: Graph-guided LLM agents for code localization. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2025.

[3] Mikhail Galkin, Xinyu Yuan, Hesham Mostafa, Jian Tang, and Zhaocheng Zhu. Towards foundation models for knowledge graph reasoning. In *International Conference on Learning Representations (ICLR)*, 2024.

[4] Paul Gauthier. Aider: Ai pair programming in your terminal. https://github.com/paul-gauthier/aider, 2023.

[5] Bernal Jiménez Gutiérrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. HippoRAG: Neurobiologically inspired long-term memory for large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[6] Taher H. Haveliwala and Sepandar D. Kamvar. The second eigenvalue of the Google matrix. *Stanford University Technical Report*, 2003.

[7] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024.

[8] Omer Levy and Yoav Goldberg. Neural word embedding as implicit matrix factorization. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 27, pages 2177–2185, 2014.

[9] Xiangyan Liu, Bo Lan, Zhiyuan Hu, Yang Liu, Zhicheng Zhang, Fei Wang, Michael Shieh, and Wenmeng Zhou. CodexGraph: Bridging large language models and code repositories via code graph databases. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics (NAACL)*, 2025.

[10] Tomas Mikolov, Ilya Sutskever, Kai Chen, Gregory S. Corrado, and Jeffrey Dean. Distributed representations of words and phrases and their compositionality. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 26, pages 3111–3119, 2013.

[11] Siru Ouyang, Wenhao Yu, Kaixin Ma, Zilin Xiao, Zhihan Zhang, Mengzhao Jia, Jiawei Han, Hongming Zhang, and Dong Yu. RepoGraph: Enhancing AI software engineering with repository-level code graph. In *International Conference on Learning Representations (ICLR)*, 2025.

[12] Larry Page, Sergey Brin, Rajeev Motwani, and Terry Winograd. The PageRank citation ranking: Bringing order to the web. *Stanford InfoLab Technical Report*, 1998.

[13] Spencer Pearson, José Campos, René Just, Gordon Fraser, Rui Abreu, Michael D. Ernst, Deric Pang, and Benjamin Keller. Evaluating and improving fault localization techniques. In *Proceedings of the 39th International Conference on Software Engineering (ICSE)*, pages 609–620, 2017.

[14] Bryan Perozzi, Rami Al-Rfou, and Steven Skiena. DeepWalk: Online learning of social representations. In *Proceedings of the 20th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 701–710, 2014.

[15] Benedek Rozemberczki, Oliver Kiss, and Rik Sarkar. Karate club: An API oriented open-source python framework for unsupervised learning on graphs. In *Proceedings of the 29th ACM International Conference on Information and Knowledge Management (CIKM)*, pages 3125–3132, 2020.

[16] Mark Weiser. Program slicing. *IEEE Transactions on Software Engineering*, SE-10(4):352–357, 1984.

[17] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Agentless: Demystifying LLM-based software engineering agents. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on the Foundations of Software Engineering (FSE)*, 2025.

[18] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 37, pages 50528–50652, 2024.

[19] Fengji Zhang, Bei Chen, Yue Zhang, Jin Liu, Daoguang Zan, Yi Mao, Jian-Guang Lou, and Lingpeng Kong. RepoCoder: Repository-level code completion through iterative retrieval and generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 2471–2484, 2023.

[20] Yuntong Zhang, Zhiyu Li, Min Lin, Denghui Zhang, Hongyu Zhang, and Lingxiao Jiang. AutoCodeRover: Autonomous program improvement. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, pages 762–774, 2024.

[21] Tao Zhou, Zoltán Kuscsik, Jian-Guo Liu, Matúš Medo, Joseph R. Wakeling, and Yi-Cheng Zhang. Solving the apparent diversity-accuracy dilemma of recommender systems. *Proceedings of the National Academy of Sciences (PNAS)*, 107(10):4511–4515, 2010.

[22] Daming Zou, Jingjing Liang, Yuyan Xiong, Michael D. Ernst, and Lu Zhang. An empirical study of fault localization families and their combinations. *IEEE Transactions on Software Engineering*, 47(2):332–347, 2021.

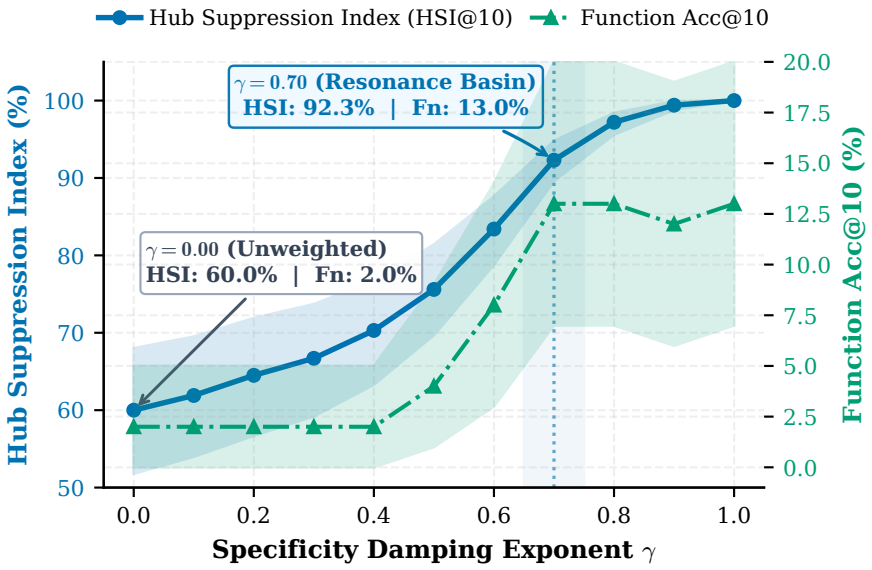


Figure 3: Specificity exponent sweep on dev_val ($N = 100$). Function Acc@10 and HSI@10 as a function of γ; $\gamma = 0$ is unweighted PPR. At the default $\gamma = 0.70$: Function Acc@10 13.0%, HSI@10 92.3%; at $\gamma = 0$: 2.0% and 60.0%.

A Supplementary Retrieval Tables

To evaluate retrieval behavior across varying degrees of textual specificity, Table 4 stratifies performance on SWE-bench Verified ($N = 344$ callable tasks) into three lexical accessibility strata: Stratum A where the gold target symbol is explicitly named in the problem description, Stratum B where the gold symbol is a direct neighbor of an issue mention, and Stratum C where the gold symbol is structurally distant without textual mention. Furthermore, Figure 3 plots the sensitivity of Function Acc@10 and HSI@10 across specificity exponent $\gamma \in [0, 1]$ on the development split ($N = 100$).

Table 4: Retrieval performance stratified by lexical accessibility on SWE-bench Verified ($N = 344$ callable tasks). Stratum A: gold target symbol is explicitly referenced in the issue description ($N = 208$). Stratum B: gold symbol is 1-hop neighbor of an issue mention ($N = 5$). Stratum C: gold symbol is structurally distant without direct mention in the issue text ($N = 131$). Top-10 hits and accuracy rates are reported.

Method	**Stratum A** ($N = 208$)	**Stratum B** ($N = 5$)	**Stratum C** ($N = 131$)	**Overall** ($N = 344$)
Okapi BM25	147 (70.7%)	0 (0.0%)	28 (21.4%)	175 (50.9%)
Standard PPR ($\beta = 0.85$)	147 (70.7%)	0 (0.0%)	25 (19.1%)	172 (50.0%)
Degree-Normalized PPR	146 (70.2%)	0 (0.0%)	24 (18.3%)	170 (49.4%)
HippoRAG Prior Scaling	142 (68.3%)	0 (0.0%)	28 (21.4%)	170 (49.4%)
Perron Static ($\gamma = 0.70$)	146 (70.2%)	0 (0.0%)	27 (20.6%)	173 (50.3%)
Perron Adaptive	146 (70.2%)	0 (0.0%)	27 (20.6%)	173 (50.3%)
Slot Fusion (RRF)	147 (70.7%)	0 (0.0%)	27 (20.6%)	174 (50.6%)